

ONEHEALTH AI

Unified Offline-First AI Screening & Care Platform

Bridging Human Health (General & Child Development) & Livestock Veterinary Care

Tailored for Rural Communities & Primary Healthcare Centers (Kopargaon, Maharashtra)

Document Version:	1.0.0 (Production Release)
Target Stack:	HTML5, CSS3, ES6+ Vanilla JS, IndexedDB, Service Worker, FastAPI, SQLite/PostgreSQL
Target Deployment:	Mobile-First Progressive Web App (PWA) + REST Backend
Target Users:	ASHA Workers, ANM Nurses, Medical Officers (MBBS/MD), Veterinary Surgeons (BVSc), Rural Farmers & Citizens
Offline Capability:	100% Autonomous (Forms, Local AI Triage, Image Store, Sync Queue)
Languages:	English, Marathi (■■■■■), Hindi (■■■■■) with Speech Assist

EXECUTIVE SUMMARY

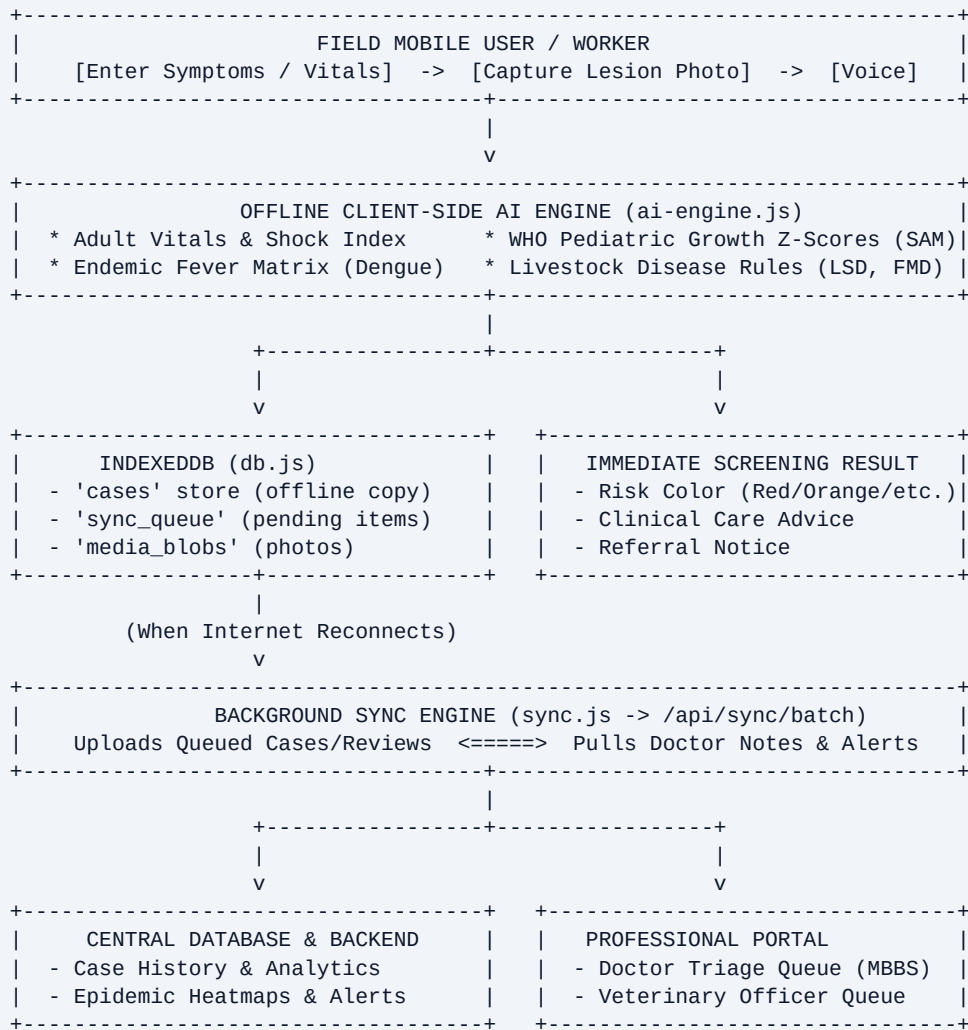
Rural communities such as Kopargaon face a twin structural crisis: a severe shortage of qualified medical doctors, pediatricians, and veterinary surgeons, paired with intermittent or non-existent cellular internet connectivity. Delays in screening common febrile illnesses, severe acute malnutrition (SAM) in under-5 children, or contagious epizootics (like Lumpy Skin Disease or Foot & Mouth Disease in dairy cattle) lead to preventable mortality, stunted development, and catastrophic economic loss for agrarian households.

ONEHEALTH AI is a production-grade, mobile-first Progressive Web Application (PWA) that empowers community health workers (ASHA/ANM) and livestock owners to perform comprehensive clinical and veterinary screening directly in the field—**entirely offline**. Using autonomous client-side expert decision engines, native IndexedDB persistence, WHO growth standard algorithms, and background queue synchronization, ONEHEALTH AI ensures uninterrupted care delivery regardless of network status.

1. System Architecture & Component Design

The system employs a client-heavy, zero-friction edge architecture. The frontend is built with pure HTML5, CSS3, and ES6+ JavaScript, requiring no heavy client bundles or compilation. The backend is an asynchronous Python FastAPI service capable of running on edge servers or cloud infrastructure.

Component Layer	Technologies Used	Core Responsibilities
Mobile PWA UI	HTML5, CSS3, ES6+ JS, Manifest, ServiceWorker	Mobile-first ergonomic interface, bottom touch navigation, speech TTS/STT, camera capture, multilingual DOM switching.
Offline Client AI	OneHealthAIEngine (Pure JS)	Autonomous offline screening for adult vitals, WHO pediatric growth Z-scores, milestone delays, and livestock disease matrices.
Offline DB Layer	IndexedDB (OneHealthDB)	Stores cases, sync queue, compressed image blobs, community outbreak alerts, and local preferences.
Sync Manager	OneHealthSyncEngine	Monitors network states (online/offline), performs batch push to /api/sync/batch and pulls server doctor reviews/alerts.
Backend API	FastAPI, Uvicorn, Pydantic v2	RESTful endpoints (/api/cases, /api/sync, /api/professionals, /api/analytics), CORS, validation, static file server.
Database	SQLAlchemy (SQLite / PostgreSQL)	Relational storage for Users, Cases, Clinical Reviews, Outbreak Alerts, and Device Sync Logs.
Professional Portal	Responsive Web View	Doctor (MBBS) & Vet (BVSc) tele-triage dashboard, risk verification, prescription submission, printable referral slips.

End-to-End Data Flow Diagram:

2. Screening Modules & Clinical Logic

A. Human General Health Triage Algorithm

Evaluates vital signs, critical red flags, and endemic differential symptoms to compute an instantaneous risk tier (GREEN, YELLOW, ORANGE, RED).

- 1. Emergency Red Flags (Immediate RED Escalation):** Severe crushing chest pain radiating to arm/jaw, sudden unilateral weakness or slurred speech (FAST stroke sign), severe resting breathlessness, altered sensorium/drowsiness, severe stiff neck with photophobia (meningism).
- 2. Vitals & Shock Index:** Shock Index = Heart Rate / Systolic BP. If Shock Index ≥ 1.0 with Systolic BP < 90 mmHg, flags immediate Haemodynamic Shock (RED). Hypoxemia is graded into Severe ($SpO_2 < 90\%$ -> RED) and Moderate ($SpO_2 90-93\%$ -> ORANGE). Hypertensive Crisis is triggered if BP $\geq 180/120$ mmHg.
- 3. Differential Endemic Matrix:** Weighs co-occurring symptom clusters for Dengue (retro-orbital headache + petechial rash + high fever), Malaria (paroxysmal rigors/chills + sweats), Typhoid (step-ladder fever + coated tongue), Pulmonary TB (productive cough > 2 weeks + night sweats + weight loss), Gastroenteritis, and Uncontrolled Diabetes (RBS > 200 mg/dL + non-healing ulcer).

B. Childhood Growth & WHO Developmental Milestones (0-5 Years)

Incorporates World Health Organization (WHO) Growth Standards and a 4-domain milestone developmental tracker.

- 1. Anthropometric Z-Score Formulas:** Computes Weight-for-Age (WAZ), Height-for-Age (HAZ), and Weight-for-Height (WHZ) relative to WHO median reference curves.
- 2. Severe Acute Malnutrition (SAM) Detector:** Triggered (RED) if: (a) Mid-Upper Arm Circumference (MUAC) < 11.5 cm, OR (b) WHZ < -3.0 SD, OR (c) Bilateral pitting pedal edema (Kwashiorkor) is present. Immediately recommends Nutritional Rehabilitation Centre (NRC) referral.
- 3. Moderate Acute Malnutrition (MAM):** Triggered (ORANGE) if MUAC is 11.5 - 12.5 cm or WHZ is -2.0 to -3.0 SD. Enrolls child into supplementary nutrition take-home ration.
- 4. 4-Domain Milestone Tracking:** Evaluates Gross Motor, Fine Motor, Language/Communication, and Social/Cognitive domains. If ≥ 2 domains exhibit delay, flags Global Developmental Delay and routes to District Early Intervention Centre (DEIC).

C. Livestock & Veterinary Health Screening Algorithm

Supports Cattle, Buffalo, Goat, Sheep, and Poultry with species-specific rectal temperature baselines and syndromic matrices.

Target Disease	Key Symptoms & Indicators	Triage Risk	Biosecurity / Clinical Action
Lumpy Skin Disease (LSD)	Cutaneous firm nodules (2-5cm), high fever ($> 104^\circ\text{F}$), milk yield crash $> 80\%$, leg edema, enlarged prescapular lymph nodes.	RED	Strict quarantine of animal, neem oil fly repellent, notify Taluka Vet Dispensary.
Foot & Mouth Disease (FMD)	Profuse ropy salivation, vesicles/blisters on gums & tongue, interdigital sores, severe lameness.	RED	Herd isolation, KMnO_4 1% mouth wash, boro-glycerine on ulcers, hoof antiseptic spray.
Acute Mastitis	Hard, hot, swollen udder quarter, milk containing clots/flakes/blood, CMT test positive.	ORANGE	Complete stripping every 3h, intramammary antibiotic infusion + systemic NSAIDs.
Black Quarter (BQ)	Crackling gas swellings over heavy muscles (thigh/shoulder), acute severe lameness, high fever.	RED	Critical emergency: High-dose Penicillin IM/IV; vaccinate remainder of herd.

Hemorrhagic Septicemia	Submandibular throat swelling, snoring breathing, respiratory stertor, high fever.	RED	Urgent IV Sulphadimidine 33.3% / Oxytetracycline administration.
PPR (Goat Plague)	Erosive stomatitis, foul watery diarrhea, oculonasal discharge, pneumonia in goats/sheep.	RED	Isolate affected small ruminants; fluid therapy + antibiotics for pneumonia.

3. Offline Storage & Synchronization Protocol

A. Native IndexedDB Data Schema

All application state resides primarily in the browser's native IndexedDB (OneHealthOfflineDB, version 2), ensuring full durability even across browser reloads or device restarts.

Store Name	Key Path	Indexes	Purpose
cases	id (String)	case_type, risk_level, village, is_synced, client_created_at	Stores complete screening records, vitals, Z-scores, photo references, triage status, and doctor reviews.
sync_queue	queue_id (Auto)	status, entity_id	Stores pending operations (SAVE_CASE, SAVE_REVIEW) waiting to be synced with backend.
media_blobs	id (String)	None	Stores compressed JPEG base64 data URLs for skin lesions, rashes, or udder swelling.
alerts	id (Integer)	None	Caches community outbreak alerts locally for offline display.
settings	key (String)	None	Stores active language, device ID, reviewer profile, and last server sync timestamp.

B. Synchronization Protocol & Conflict Handling

Synchronization is bidirectional, fault-tolerant, and atomic:

- 1. Event-Driven Auto Sync:** The engine registers `window.addEventListener('online')`. When connectivity is restored, it initiates an automatic batch flush.
- 2. Batch Sync Endpoint (/api/sync/batch):** Collects all pending records in a single payload. The server validates, updates existing records, registers clinical reviews, and returns newly created server cases and active outbreak alerts.
- 3. Queue Clearing & State Reconciliation:** On HTTP 200 response, queued items are atomically removed from IndexedDB, and cases are marked with `is_synced: true`. Server-side clinical reviews and prescription updates are upserted into the local database.

4. Multilingual Engine & Accessibility

To serve rural populations with varying literacy levels, ONEHEALTH AI includes built-in accessibility features:

- 1. Trilingual Support:** Full translations for **English**, **Marathi (मराठी)**, and **Hindi (हिन्दी)**. Language switching is instantaneous via DOM data attributes without requiring page reload.
- 2. Text-to-Speech (TTS) Narration:** Uses the browser Web Speech API (`SpeechSynthesisUtterance`) tuned with regional voice codes (mr-IN, hi-IN, en-IN) and a deliberate 0.95x cadence for clear comprehension by rural villagers.
- 3. Voice Dictation (STT):** Leverages SpeechRecognition allowing ASHA workers or farmers to speak symptoms and clinical notes directly into forms.
- 4. On-Device Image Compression:** Resizes camera photos on an HTML5 canvas to a max 1024px dimension and 75% JPEG quality (~40-80 KB), calculating an Erythema Redness Index while preventing 2G/3G sync timeouts.

5. Complete Project Directory Structure

```
proud-turing/
├── backend/
│   ├── __init__.py
│   ├── main.py
│   ├── database.py
│   ├── models.py
│   ├── schemas.py
│   ├── seed_data.py
│   └── routers/
│       ├── cases.py
│       ├── sync.py
│       ├── professionals.py
│       └── analytics.py
├── public/
│   ├── index.html
│   ├── manifest.json
│   ├── service-worker.js
│   ├── css/
│   │   └── main.css
│   └── js/
│       ├── db.js
│       ├── ai-engine.js
│       ├── sync.js
│       ├── i18n.js
│       ├── voice.js
│       ├── camera.js
│       ├── analytics.js
│       ├── app.js
│       └── icons/
├── tests/
│   └── test_api.py
├── run.py
└── README.md
```

FastAPI App entrypoint, CORS & static mounting
 # SQLAlchemy engine (SQLite & PostgreSQL support)
 # User, Case, ClinicalReview, OutbreakAlert models
 # Pydantic v2 validation schemas & batch payloads
 # Realistic seed generator for Kopargaon cases & alerts
 # Cases CRUD & filtering API
 # Batch sync & offline queue reconciliation endpoint
 # Doctor (MBBS) & Vet (BVSc) triage queues & reviews
 # Community surveillance & outbreak alerts API
 # Progressive Web Application (PWA) Frontend
 # Single Page Application HTML5 structure
 # PWA Manifest (standalone display, theme, icons)
 # Service Worker with Cache-First app shell caching
 # Mobile-first CSS design system & print styles
 # Promisified IndexedDB offline database layer
 # Autonomous client-side AI triage engine
 # Network monitor & batch sync coordinator
 # English, Marathi, Hindi translation dictionaries
 # Speech synthesis (TTS) & dictation (STT)
 # Camera capture, canvas compression & color index
 # Epidemiological surveillance & chart visualizer
 # Main UI router, form handlers & review modals
 # PWA icons (icon.svg, icon-192.png, icon-512.png)
 # Automated test suite (6 passing test suites)
 # Convenient one-click server starter script
 # Complete project README and usage guide

6. Installation, Execution & Testing Guide

A. Prerequisites & Dependencies

Ensure Python 3.10+ is installed. Install required packages:

```
pip install fastapi uvicorn sqlalchemy pydantic pillow reportlab
```

B. Starting the Server

Execute the server runner script:

```
python run.py
```

The server initializes the database, automatically seeds realistic Kopargaon demonstration cases and active outbreak alerts, and serves both the PWA and REST APIs on **http://localhost:8000**.

C. Running Automated Unit Tests

Execute the comprehensive test suite:

```
python -m unittest tests/test_api.py
```

D. Step-by-Step Offline & Sync Verification

- 1. Open Browser:** Navigate to <http://localhost:8000> in Chrome/Edge.
- 2. Switch to Offline Mode:** Press F12 -> *Network tab* -> Select '**Offline**'.

- 3. Complete a Screening:** Tap *Start Screening* -> Select Human, Child, or Livestock -> Submit.
- 4. Verify Offline Result:** Observe instant AI risk calculation and recommendations. In the *Cases* tab, notice the record marked as ■ Saved Offline.
- 5. Switch Back to Online:** In DevTools, set Network to '**No throttling (Online)**'. Watch the sync badge turn green and the record status update to ■ Synced.
- 6. Professional Portal Sign-Off:** Navigate to the *Portal* tab -> Open any case -> Add clinical prescription -> Tap *Submit Review & Sign-Off* -> Print clinical referral slip.